SkinAnalysis AI Skin Care Analyzer

Project Report and Technical Summary

Prepared on: February 26, 2026
Project type: Full-stack AI web application
Purpose: Internship submission and college project documentation

---

1. Executive Summary

SkinAnalysis AI is an end-to-end skin analysis platform that predicts facial skin type (`dry`, `normal`, `oily`) from uploaded images and returns explainable, personalized skincare guidance.
The application combines secure authentication, ML inference, Grad-CAM explainability, user history tracking, and a responsive dashboard interface.

This report is based on a full repository audit and runtime verification of: - Backend routes and API behavior - Database schema and live SQLite snapshot - ML data pipeline, training script, and evaluation output - Frontend UI/UX implementation and dashboard navigation - Deployment setup (local + Docker)

---

2. Problem Statement

Users often misidentify their skin type and choose unsuitable products.
This project solves that by providing: - AI-based skin type classification - Explainable prediction output (Grad-CAM) - Structured skin-type-specific recommendations - Persistent user history with progress records

---

3. Objectives Completed

1. Built a production-style Flask web application around AI inference.
2. Implemented secure signup/login/logout with session management.
3. Enforced CSRF protection for forms and APIs.
4. Added robust image upload validation and storage safety.
5. Integrated model inference with confidence and class probabilities.
6. Added Grad-CAM explainability generation and fallback handling.

7. Persisted per-user analysis records in SQLite with history CRUD.
8. Delivered a responsive dashboard with in-page multi-view UX.
9. Added data cleaning/splitting, training, and evaluation scripts.
10. Added account-scoped schedule management with backend CRUD APIs.
11. Containerized deployment with Docker and Gunicorn.

---

4. End-to-End System Workflow

4.1 Analysis Workflow

1. User authenticates in the app.
2. User uploads a face image from dashboard analyzer.
3. Backend validates extension, MIME, content, and size.
4. Image is preprocessed and passed to the trained model.
5. Model outputs skin type and confidence scores.
6. Grad-CAM heatmap is generated for explainability.
7. Result and metadata are stored in the `results` table.
8. Dashboard updates analysis output and history records.

4.2 Schedule Workflow

1. User opens schedule view from dashboard sidebar.
2. Frontend fetches events from `GET /api/schedule/events`.
3. Create/update/delete actions call schedule mutation APIs with CSRF token.
4. Backend validates payload and enforces user ownership.
5. Records are persisted in `schedule_events` table and returned to UI.
6. Updated schedule is rendered consistently across devices for same account.

---

5. Architecture Overview

```
Browser (HTML templates + JS + CSS)
   |
   | HTTP / JSON API
   v
Flask App (app.py)
   |-- Auth + Session + CSRF
   |-- Upload validation
   |-- Inference orchestration
   |-- History and dashboard context
   |
   +--> ML Layer
   |      preprocessing/image_preprocess.py
   |      preprocessing/gradcam.py
   |      model/skin_model.keras
   |
   +--> Recommendation Layer
   |      recommender/products.py
   |
   +--> Data Layer
          database/db.py
          database/auth.py
          database/save_result.py
          database/schedule_events.py
          database/skin_care.db
```

---

6. Tech Stack

- Backend: Flask, Gunicorn
- ML/Data: TensorFlow/Keras, NumPy, OpenCV, scikit-learn, pandas
- Image handling: Pillow, OpenCV
- Database: SQLite
- Frontend: Server-rendered templates + JavaScript
- Explainability: Grad-CAM
- Deployment: Docker, Docker Compose

Dependencies from `requirements.txt`:
`flask`, `gunicorn`, `numpy`, `pandas`, `opencv-python`, `tensorflow`, `mediapipe`,

`scikit-learn`, `pillow`, `matplotlib`

---

7. Repository Module Responsibilities

- `app.py`
  Core Flask app, route handlers, CSRF/session security, upload/inference flow, dashboard/history integration.
- `database/`
  DB initialization, schema resilience, auth queries, result save/fetch/delete logic, schedule event CRUD.
- `preprocessing/`
  Image preprocessing, Grad-CAM generation, optional face/skin extraction helpers.
- `model/`
  Model code and artifacts (`skin_model.keras`, `skin_model_best.keras`, `class_names.json`).
- `recommender/`
  Skin-type recommendation content and helper logic.
- `templates/`
  UI pages for home, dashboard, auth, settings, and result rendering.
- `static/js/upload.js`
  Upload API calls, progress UI, result rendering, history updates, view switching.
- `static/js/schedule.js`
  Schedule event rendering, filters/details, backend API sync, and reminder export (`.ics`).
- `static/css/`
  Responsive styling and dashboard visual system.
- `prepare_dataset.py`
  Data quality filtering, deduplication, deterministic train/val/test split.
- `train.py`
  Transfer-learning training pipeline.
- `evaluate_model.py`
  Test evaluation and class-wise metrics.
- `Dockerfile`, `docker-compose.yml`
  Runtime containerization.

---

8. Backend and Security Implementation

8.1 Session and CSRF Security

- `SESSION_COOKIE_HTTPONLY = True`
- `SESSION_COOKIE_SAMESITE = "Lax"`
- `SESSION_COOKIE_SECURE` controlled by environment variable
- CSRF token created per session and validated on all mutating methods (`POST`, `PUT`, `PATCH`, `DELETE`)
- CSRF accepted via form field or `X-CSRF-Token` header
- Protected routes are user-authenticated

8.2 Upload Validation and Safety

- Allowed extensions: `.jpg`, `.jpeg`, `.png`, `.bmp`, `.webp`
- MIME and real content verification
- Max upload size: `5 MB` (`MAX_CONTENT_LENGTH`)
- Unique file naming with safe extension handling

8.3 Inference Metadata Logging

For every scan, backend records: - Predicted class - Confidence score - Class probabilities JSON - Low-confidence flag - Inference latency (`inference_ms`) - Model version - Original image path and Grad-CAM path

8.4 Schedule API Validation and Ownership Controls

- Schedule event type is validated against allowed enum (`scan`, `appointment`, `reminder`, `refill`)
- Priority is validated against allowed enum (`low`, `medium`, `high`)
- Datetime, title length, description length, and reminder bounds are validated server-side
- Mutating schedule APIs require valid CSRF token
- All schedule reads/writes/deletes are scoped by authenticated `user_id`

---

9. Route and API Inventory

9.1 Web Routes

- `GET /`
- `GET|POST /signup`

- `GET|POST /login`
- `GET /logout`
- `GET /analyzer`
- `GET /settings` (dashboard settings view redirect)
- `GET /history` (dashboard history view redirect)
- `GET /history/<result_id>`
- `POST /history/<result_id>/delete`
- `POST /analyze`

9.2 API Routes

- `GET /api/csrf-token`
- `POST /api/auth/signup`
- `POST /api/auth/login`
- `GET /api/auth/me`
- `POST /api/auth/logout`
- `GET /api/results?limit=...`
- `GET /api/schedule/events`
- `POST /api/schedule/events`
- `PATCH /api/schedule/events/<event_id>`
- `DELETE /api/schedule/events/<event_id>`
- `POST /api/analyze`

---

10. Database Design and Current Snapshot

10.1 Core Tables

- `users`
- `id`, `name`, `email` (unique), `password_hash`, `created_at`
- `results`
- `id`, `user_id`, `skin_type`, `confidence`, `image_path`
- `gradcam_image_path`, `model_version`, `class_probabilities_json`
- `is_low_confidence`, `inference_ms`, `created_at`
- `products`
- seed recommendations per skin type
- `schedule_events`
- `id`, `user_id`, `title`, `event_type`, `priority`, `event_datetime`
- `description`, `reminder_minutes`, `is_completed`, `created_at`, `updated_at`

Schema resiliency in `database/db.py`: - Adds missing `results` columns at startup if needed - Ensures index `idx_results_user_id_created_at` - Ensures `schedule_events` table and indexes: - `idx_schedule_events_user_datetime` - `idx_schedule_events_user_created_at`

10.2 Live SQLite Snapshot (Verified on February 26, 2026)

- `users`: 3
- `results`: 41
- `products`: 9
- `schedule_events`: 1
- Result date range: `2026-02-06 07:32:04` to `2026-02-26T10:56:49+05:30`

---

11. Data Preparation and ML Pipeline

11.1 Dataset Cleaning and Split (`prepare_dataset.py`)

- Source root: `dataset/train`
- Quality thresholds:
- minimum width: 80
- minimum height: 80
- minimum sharpness: 20.0 (Laplacian variance)
- Duplicate filtering by hash and copy-name pattern detection
- Reproducible split (seed=42):
- train: 70%
- val: 15%
- test: 15%
- Output report: `dataset/split/cleaning_report.json`

11.2 Post-Cleaning Dataset Statistics

- dry: scanned 652, kept 648
- normal: scanned 1104, kept 1053
- oily: scanned 1000, kept 651

Split counts: - train: dry 453, normal 737, oily 455 - val: dry 97, normal 157, oily 97 - test: dry 98, normal 159, oily 99

11.3 Training Pipeline (`train.py`)

- Input shape: `224 x 224`

- Backbone: MobileNetV2
- Two-stage training: 1. classifier head training 2. partial backbone fine-tuning
- Augmentations include rotation, zoom, translation, flip, contrast
- Class weights derived from class frequency
- Callback strategy:
- best-model checkpoint
- early stopping
- learning-rate reduction on plateau

Saved artifacts: - `model/skin_model.keras` - `model/skin_model_best.keras` - `model/class_names.json`

11.4 Evaluation Results (`evaluate_model.py`)

Evaluation executed on February 21, 2026 using `dataset/split/test` (356 samples): - Overall accuracy: `51.97%` - Confusion matrix (`true x predicted`): - `[[54, 25, 19], [31, 91, 37], [20, 39, 40]]` - Per-class performance: - dry: precision 51.43%, recall 55.10%, F1 53.20% - normal: precision 58.71%, recall 57.23%, F1 57.96% - oily: precision 41.67%, recall 40.40%, F1 41.03%

---

12. Explainability and Recommendation Logic

12.1 Grad-CAM Explainability

`preprocessing/gradcam.py` uses robust fallback strategies: - Graph-based Grad-CAM path - Manual forward-pass fallback - Strict fallback for sequential/checkpoint edge cases - Safe fallback output when gradients are unavailable

This improves reliability of visual explanation in production flows.

12.2 Recommendation Engine

`recommender/products.py` provides structured skin-type advice: - Description and characteristics - Do and don't guidance - Product recommendations - Morning/evening routine suggestions

Legacy-compatible helpers remain in: - `recommender/recommend.py` - `recommender/product_rules.py`

---

13. Frontend and User Experience

13.1 User Flows

- Landing page with app overview and auth access
- Login/signup forms with clear validation feedback
- Dashboard with analyzer, routine, schedule, history, and settings views
- In-page history rendering without full page switch
- In-page schedule management with add/edit/complete/delete actions
- Result detail and history record deletion actions

13.2 Client-Side Logic (`static/js/upload.js`)

- Asynchronous upload requests with progress indicator
- Dynamic result rendering after inference
- History insertion, trend calculations, and deletion modal
- Hash-based dashboard view switching (`home` , `routine` , `schedule` , `history` , `settings`)
- CSRF token fetch/refresh integration for protected API calls

13.3 Schedule Client Module (`static/js/schedule.js`)

- Fetches account-scoped schedule events via REST API
- Performs create/update/delete with CSRF-protected calls
- Renders summary cards, filters, event list, and detail panel
- Exports event reminders as `.ics` files for phone calendar import

13.4 Styling System

- `static/css/base.css` : shared shell/layout system
- `static/css/home.css` : landing page
- `static/css/analyzer.css` : dashboard + analyzer + history
- `static/css/auth.css` , `static/css/result.css` , `static/css/settings.css` : page-specific styling

Responsive behavior is implemented for desktop, tablet, and mobile breakpoints.

---

14. Deployment and Operations

14.1 Local Run

1. Create venv and install dependencies.
2. Set environment variables: - `SECRET_KEY` (required) - `SESSION_COOKIE_SECURE` (`0` local, `1` HTTPS deployments)
3. Run `python app.py`.

14.2 Docker Run

- Base image: `python:3.10-slim`
- Runtime includes OpenCV system dependencies (`libgl1`, `libglib2.0-0`)
- Served via Gunicorn on port `5000`
- Compose mounts runtime asset folders:
- `static/uploads`
- `static/gradcam`

---

15. Major Work Completed (Internship Contribution Summary)

1. Designed and implemented secure authentication and session control.
2. Implemented CSRF protection across form and API mutation flows.
3. Built a strict image validation + upload processing pipeline.
4. Integrated TensorFlow inference with complete metadata logging.
5. Implemented reliable Grad-CAM explainability generation.
6. Built user-scoped history management with secure ownership checks.
7. Implemented dashboard-first UX with in-page navigation and async updates.
8. Added backend-synced schedule management across devices/accounts.
9. Added structured skincare recommendation engine integration.
10. Developed dataset cleaning and deterministic splitting workflow.
11. Built transfer-learning training and evaluation scripts.
12. Containerized the application for reproducible deployment.

---

16. Engineering Challenges and Resolutions

- Challenge: Untrusted image uploads can break inference or expose attack surface. Resolution: Added extension + MIME + actual content verification and size capping.

- Challenge: CSRF risk in hybrid HTML/API app.
  Resolution: Unified token lifecycle and validation across web and API mutation routes.
- Challenge: Grad-CAM may fail on some model graph structures.
  Resolution: Implemented multi-path fallback logic to keep explainability output available.
- Challenge: Data quality inconsistencies in raw dataset.
  Resolution: Added automated filtering, duplicate suppression, and deterministic split generation.
- Challenge: Maintaining dashboard continuity while switching functional views.
  Resolution: Implemented hash-driven in-page view switching and dynamic rendering in JS.
- Challenge: Keeping schedule data consistent across devices and sessions.
  Resolution: Replaced browser-local event persistence with authenticated backend CRUD APIs scoped by `user_id`.

---

17. Current Limitations

- Model test accuracy is currently moderate (`51.97%`), especially weaker on `oily` recall.
- Automated tests (unit/integration/e2e) are limited and should be expanded.
- Advanced production controls (rate limiting, centralized logging, secrets manager) can be strengthened.
- Schedule reminders are exported as `.ics`; native push notifications are not implemented server-side.
- Some legacy templates/utilities still exist and can be consolidated.

---

18. Next Improvement Roadmap

1. Improve class balance and data quality, especially for `oily` samples.
2. Benchmark alternative backbones (EfficientNet/ConvNeXt variants).
3. Add model experiment tracking and model-card style documentation.
4. Add automated QA pipeline (backend + frontend + API).
5. Add native PDF export of report/history from the web app.

---

19. College Viva and Company Discussion Points

1. Why Flask with server-rendered templates for fast delivery.
2. Security architecture choices in a mixed HTML + API system.
3. Upload validation and trust boundaries for image inputs.
4. Explainability strategy and Grad-CAM fallback reliability.
5. Data cleaning pipeline and reproducibility choices.
6. Schedule sync design (local UI state vs backend event persistence).
7. Training/evaluation interpretation and optimization strategy.
8. Full-stack deployment tradeoffs (local vs containerized).

---

20. Conclusion

SkinAnalysis AI demonstrates a complete AI product lifecycle implementation: data preparation, model training, explainable inference, secure full-stack integration, persistent user history, backend-synced scheduling, and deployable delivery.
The project is ready for internship review and academic presentation, with a clear roadmap for performance and production maturity improvements.

If required, this report can be converted into PDF for official submission.